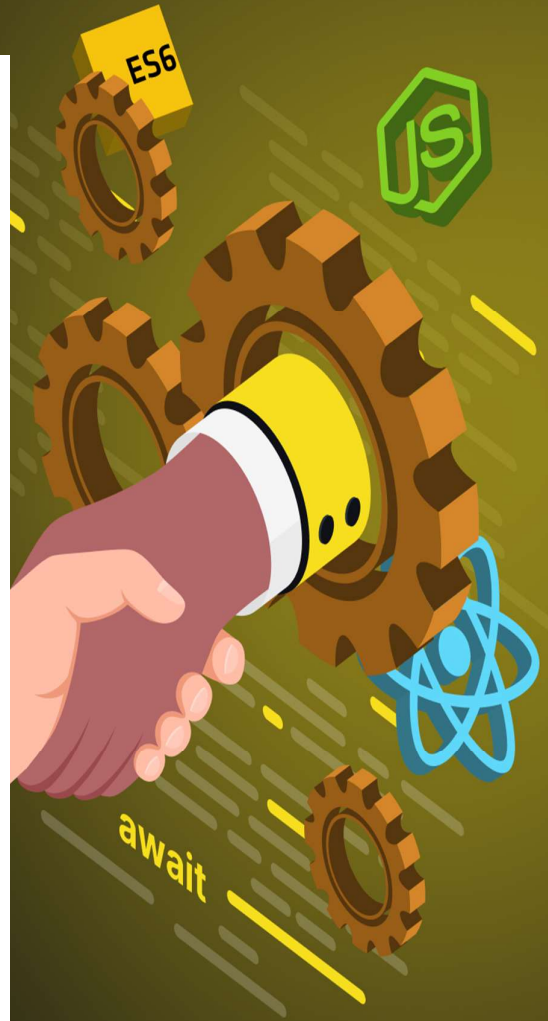


JavaScript-05

Async/Await och Axios



30 september 2026

JavaScript – utan tjafs
Författare: Acke Strömberg



För en kanin

Async/Await och Axios

Async/Await - historik

Problematik

För att se det stora sammanhanget kan man sammanfatta JavaScripts resa så här:

1. **Callbacks** (Det gamla sättet): Funktioner inuti funktioner inuti funktioner. Koden blir en sidleds-pyramid (**callback hell**).
2. **Promises (.then / .catch)**: Löste pyramiden genom att låta oss kedja ihop saker på rad istället. Men vi var fortfarande tvungna att använda mycket specialfunktioner och return.
3. **Async / Await (Det moderna sättet)**: Det slutgiltiga steget. Vi skriver koden helt platt, uppifrån och ner, precis som vanlig kod. JavaScript sköter allt det krångliga i bakgrunden.

JavaScript körs på en enda tråd (eng. thread). Det betyder att språket bara kan göra en sak åt gången. Om JavaScript skulle stanna upp och vänta i 3 sekunder på att en bild laddas eller att en databas svarar, då skulle hela webbsidan frysa under tiden. Användaren skulle inte ens kunna klicka på en knapp.

För att lösa detta använder JavaScript **callbacks** (återanrop). En callback är helt enkelt en funktion som skickas med som ett argument till en annan funktion, med instruktionen: *"Gör det här i bakgrunden, och när du är helt klar, kör den här funktionen då."* Man brukar ofta förkorta en callback med **cb**, ett exempel på en sådan callback visas i bild 1.

```
// En funktion som tar en callback som argument
function hämtaAnvändare(userId, callback) {
  fetch(`https://typicode.com{userId}`)
    .then(res => res.json())
    // Här körs din callback när datan är klar
    .then(data => callback(data));
}

// Vi kör funktionen och skickar in callbacken som en anonym arrow function
hämtaAnvändare(1, data => console.log("Hittade användare:", data.name));
```

Bild 1 Ett fetch-anrop i funktionen hämtaAnvändare med en callback-funktion som argument.

I bild 1 så börjar det med att fetch startar ett anrop till en endpoint i REST API:et, asynkront i bakgrunden. När det kommer tillbaka ett promise som är lyckat hoppar koden in i den första .then() där svaret omvandlas till ett läsbart JavaScript-objekt. När det är klart hoppar koden in i den andra .then() där den går till den callback-funktion som du skickade med i anropet till hämtaAnvändare().

Callback hell

Tänk dig att du ska baka en tårta, men du måste göra allt i exakt rätt ordning och varje steg tar tid. I början av JavaScripts historia fanns varken Promises eller async/await. Då var man tvungen att nästla (lägga inuti varandra) dessa funktioner. I bild 2 kan man se något som kallas **callback hell**.

```
// Vi vill köpa ingredienser -> blanda smet -> baka -> dekorera
köpIngredienser(function(ingredienser) {

    // När ingredienserna är köpta, gör detta:
    blandaSmet(ingredienser, function(smet) {

        // När smeten är blandad, gör detta:
        bakaTårta(smet, function(tårta) {

            // När tårtan är bakad, gör detta:
            dekoreraTårta(tårta, function(klarTårta) {
                console.log("Tårtan är äntligen klar att ätas!");
            });
        });
    });
});
```

Bild 2 Läger till en eventListener till knappens click event.

När du tittar på koden ovanför ser du att den bildar en pil eller en trappa som rör sig längre och längre in mot höger. Detta kallas för **Pyramid of Doom**.

För en nybörjare (och för erfarna utvecklare!) blir detta snabbt en mardröm av tre anledningar:

1. **Svårläst:** Du måste läsa koden inifrån och ut, istället för uppifrån och ner.
2. **Parentes-kaos:** Titta längst ner i koden: }); }); });. Det är extremt lätt att missa en slutparentes eller måsvinge, vilket kraschar hela programmet.

-
3. **Felhantering:** Om något går fel (t.ex. äggen är slut i affären på steg 1), måste du skriva felhantering inuti *varje enskild* funktion. Koden blir enorm och rörig.

När JavaScript-utvecklare tröttnade på callback-helvetets alla nästlade parenteser uppfann de **Promises** (löften). Det introducerades som en räddning för att platta till koden så att den läses uppifrån och ner, snarare än snett inåt höger.

Promise

Tänk dig att du går till en snabbmatsrestaurang och beställer en burgare.

1. Istället för att du måste stå kvar vid kassan och vänta tills maten är klar (vilket skulle blockera kön), ger kassören dig en **elektronisk puck** (en pager).
2. Den här pucken är ett **Promise** (ett löfte). Den säger: *"Just nu har du ingen mat, men jag lovar att du kommer få den så fort den är klar."*
3. Du kan gå och sätta dig, prata med din kompis eller scrolla på mobilen. Du är inte blockerad.
4. När maten är klar börjar pucken blinka och vibrera. Löftet är uppfyllt! Du går och byter ut pucken mot din burgare. Om ingredienserna visar sig vara slut piper pucken rött – löftet bröts.

I JavaScript fungerar ett Promise exakt likadant. Det är ett **objekt** som representerar ett asynkront värde som inte finns tillgängligt just nu, men som kommer att finnas i framtiden.

Ett löfte kan bara befinna sig i ett av tre lägen:

- **Pending (Väntar):** Hamburgaren tillagas. Operationen pågår i bakgrunden.
- **Fulfilled (Uppfyllt):** Hamburgaren är klar! Allt gick bra. (Här triggas `.then()`).
- **Rejected (Avvisat):**

Hamburgaren brände fast eller köttet är slut. Något gick fel. (Här triggas `.catch()`).

Chaining `.then` `.catch`

Istället för att stoppa funktioner *inuti* varandra, tillåter Promises oss att lägga dem **på rad** efter varandra med hjälp av ordet `.then()` (vilket betyder *"och sedan"*).

Om vi tar vårt tårt-exempel från tidigare och skriver om det med Promises, förvandlas callback-helvetet till en snygg, rak **metodkedja**:

```
// Varje funktion returnerar nu en "elektronisk puck" (ett Promise)
köpIngredienser()
  .then(ingredienser => {
    return blandaSmet(ingredienser); // Gå till nästa steg
  })
  .then(smet => {
    return bakaTårta(smet); // Gå till nästa steg
  })
  .then(tårta => {
    return dekoreraTårta(tårta); // Gå till sista steget
  })
  .then(klarTårta => {
    console.log("Tårtan är klar!");
  })
  .catch(fel => {
    // EN gemensam säkerhetslina för ALLA steg ovanför!
    console.error("Något gick fel under tårtbaket:", fel);
  });
```

Bild 3 Exempel på en rak metodkedja.

1. **Koden läses uppifrån och ner:** Ingen pyramid som vandrar åt höger. Det är mycket lättare för det mänskliga ögat att följa flödet.
2. **Gemensam felhantering (.catch):** Du behöver inte skriva felhantering för varje enskild funktion. Om affären har stängt i steg 1, eller om ugnen går sönder i steg 3, så avbryts kedjan omedelbart och koden hoppar direkt ner till .catch() i botten.
3. **Inga nästlade parenteser:** Slut på }); }); });-kaoset

Bättre, men inte optimalt

Även om Promises var en fantastisk förbättring, insåg utvecklare efter några år att det fortfarande fanns problem. Koden blev full av .then(), pilfunktioner (=>) och krav på att man *måste* komma ihåg ordet return på varje rad (annars bröts kedjan). Det kändes fortfarande inte som "normal", vanlig JavaScript.

Det var då man tog nästa steg och skapade **Async / Await** – vilket gör att vi kan skriva exakt samma logik men helt utan .then() och .catch().

Async/Await - förklaring

Vår egen robotassistent

Om **Promises** är själva den elektroniska pucken du får i handen, så är **async/await** sättet vi instruerar JavaScript att hantera pucken på, så att koden blir så lik verkliga livet som möjligt.

Tänk dig att du har en magisk robotassistent som ska hämta maten åt dig. Istället för att du själv måste hålla koll på pucken, programmerar du roboten. Här är exempel på hur restaurang koden ser ut med async/await.

```
// Vi skapar en asynkron funktion (roboten) som ska sköta hela processen
async function hämtaMiddagsBeställning() {
  console.log("Roboten går till kassan och beställer...");

  try {
    // 1. Beställ burgaren. await -> roboten stannar här och väntar på pucken.
    const burgare = await beställBurgare();
    console.log("Burgaren är klar! Roboten tar emot den.");

    // 2. Beställ franskisar. Roboten väntar tills franskisarna är färdiga.
    const pommes = await beställPommes(burgare);
    console.log("Franskisarna är klara!");

    // 3. Beställ dricka. Roboten väntar tills drickan är färdig.
    const dricka = await beställDricka();
    console.log("Drickan är upphälld!");

    // Allt gick bra! Roboten serverar maten.
    console.log("Middagen är serverad: Burgare, pommes och dricka!");
  } catch (fel) {
    // Om pucken börjar pipa rött vid NÅGOT av stegen hamnar vi direkt här.
    console.error("Restaurangen misslyckades med beställningen:", fel);
  }
}

// Vi startar igång processen
hämtaMiddagsBeställning();
console.log("Jag gör andra saker medan roboten väntar på maten!");
```

Bild 4 Restaurangen med async/await.

Teorin

För att förstå varför detta är så genialt måste vi titta på vad de två nyckelorden faktiskt gör rent tekniskt:

1. `async`

När du sätter `async` framför en funktion (async function), berättar du för JavaScript-motorn att: *"Den här funktionen kommer att innehålla steg som tar tid. Kör den i bakgrunden."*

- **Teoretisk effekt:** En `async`-funktion returnerar *alltid* ett Promise automatiskt, oavsett vad du skriver inuti den. Funktionen blir som en egen liten isolerad bubbla i ditt program.

2. `await`

Det är här magin sker. Du kan bara använda ordet `await` **inuti** en `async`-funktion. När JavaScript ser `await beställBurgare()`, händer följande:

- **Funktionen pausas:** JavaScript-motorn stannar upp exekveringen *inuti* just den här funktionen. Den väntar på att den "elektroniska pucken" (promiset) ska gå från läget *Pending* till *Fulfilled*.
- **Huvudtråden är fri:** Det fantastiska är att resten av din webbplats eller applikation **inte fryser**. JavaScript-motorn lämnar tillfälligt `async`-bubblan och fortsätter att köra annan kod under tiden (vilket bevisas av att vår sista `console.log` i exemplet ovan körs direkt, utan att vänta på maten).
- **Värdet packas upp:** När maten är klar "packar `await` upp" promiset och sparar resultatet direkt i variabeln `burgare`. Ingen `.then()` behövs!

3. `try...catch` (Standardiserad felhantering)

I det gamla Promise-sättet använde vi `.catch()`. Med `async/await` är vi tillbaka till helt vanlig JavaScript-programmering.

- Om ett löfte blir **Rejected** (avvisat) inuti ett `try`-block, kastar JavaScript ett vanligt fel. Exekveringen avbryts direkt och koden hoppar omedelbart till `catch`-blocket. Det fungerar exakt likadant som när du hanterar fel i helt vanlig, synkron kod
-

Sammanfattning

- **Callbacks:** Du måste stå kvar vid kassan. När maten är klar måste du ställa dig i nästa kö för pommes, och i nästa kö för dricka. Du blir låst i en lång, krånglig kedja på plats.
- **Promises (.then):** Du får elektroniska puckar. Du kan gå och sätta dig, men du måste programmera instruktioner på varje puck: *"När denna piper, gör detta, och ge mig nästa puck..."* (.then().then()).
- **Async/Await:** Du skickar en robot (async). Roboten står och väntar tålmodigt vid varje steg (await) medan du själv kan luta dig tillbaka. Koden blir spikrak, läses uppifrån och ner, och är extremt lätt att förstå.

Async/Await – avancerad kod

Parallella beställningar

I vårt förra exempel gjorde roboten allt i en lång sekvens:

```
vänta på burgaren → → vänta på franskisar → → vänta på drickan.
```

Bild 5 Mycket väntan blir det.

Om varje sak tar 10 minuter tar hela middagen **30 minuter** att få. Men i verkligheten kan kockarna i köket göra burgaren, friterar pommesen och hälla upp drickan **samtidigt**. För att göra detta i JavaScript använder vi **Promise.all()**. Det gör att alla asynkrona processer startar samtidigt, och vi väntar bara på att **alla** ska bli klara. Total tid blir då bara 10 minuter (tiden för den långsammaste rätten).

```
async function hämtaSnabbMiddagParallell() {
  console.log("Roboten skickar in alla beställningar SAMTIDIGT...");

  try {
    // Vi startar alla tre Promises på samma gång (utan 'await' framför varje)
    const löfteBurgare = beställBurgare();
    const löftePommes = beställPommes();
    const löfteDricka = beställDricka();

    // Nu väntar vi på att ALLA tre ska bli klara.
    // Promise.all tar en array med löften och "packar upp" dem i en ny array.
    const [burgare, pommes, dricka] = await Promise.all([
```

```
    löfteBurgare,  
    löftePommes,  
    löfteDricka  
  ]);  
  
  console.log(`Middagen är klar på rekordtid! Serverar: ${burgare}, ${pommes}  
och ${dricka}.`);  
  
  } catch (fel) {  
    // OBS! Om ETT ENDA av löftena i Promise.all misslyckas (avvisas),  
    // så avbryts allt direkt och koden hoppar hit.  
    console.error("Middagen misslyckades eftersom något i köket strulade:",  
fel);  
  }  
}
```

Bild 5 Restaurang med Promise.all().

I vårt förra exempel fick roboten vänta på burgaren, *sedan* beställa pommes och vänta på dem, och *sedan* dricka. Det tog onödigt lång tid. Om köket kan göra allt samtidigt vill vi skicka iväg alla beställningar på en och samma gång!

Det gör vi med **Promise.all()**. Vi startar alla asynkrona processer samtidigt, och använder sedan `await Promise.all(...)` för att pausa koden tills *alla* puckar blinkar grönt. Om en enda puck piper rött, fångas felet direkt i vår `catch`.

Axios

Smidigare fetch

Hittills har vi använt webbläsarens inbyggda `fetch()` tillsammans med `.then()` och `.catch()` för att hantera asynkrona anrop. Det fungerar, men när man ska göra flera anrop efter varandra blir koden lätt kapslad och svårläst (något som utvecklare kallar för *.then-sallad*). I det här kapitlet ska vi titta på hur modern JavaScript löser detta med nyckelorden `async` / `await`, samt lära oss att använda branschens populäraste API-verktyg, **Axios**.

Varför byta

`fetch()` är inbyggt i webbläsaren, vilket är skönt, men det har två stora nackdelar i verkliga projekt:

1. **Den dolda JSON-omvandlingen:** `fetch()` 给 ger dig bara ett rått svar (ett ström-objekt). Du måste *alltid* köra `.json()` för att kunna läsa datan. Axios fattar att du vill ha JSON och gör detta automatiskt i bakgrunden.
2. **Den "falska" tryggheten vid fel:** Om du gör ett `fetch`-anrop till en länk som inte finns (HTTP-status 404 Not Found) eller om servern har kraschat (500), tycker `fetch()` ändå att anropet "gick bra" (eftersom du fick nätverkskontakt). Det betyder att koden **inte** hoppar till ditt `catch`-block automatiskt! Du måste själv skriva `if (!response.ok)`. Axios däremot kastar ett riktigt fel vid alla dåliga statuskoder, så att koden direkt hoppar till `catch`.

Nedladdning

För att kunna köra med Axios i en vanlig enkel JavaScript-kod med `index.html`-fil så behöver man lägga till kodraden i `head`-taggen.

```
<script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
```

Bild 6 Lägg till Axios i en html-fil.

I ett modernt projekt som körs med **Vite och React** använder vi inte CDN-länkar. Istället installerar vi biblioteket lokalt i vårt projekt via terminalen.

```
npm install axios
```

Bild 7 Lägg till Axios i ett JavaScript-projekt, till exempel React JS.

Implementering i React-kod

Här kombinerar vi Axios med Reacts `useEffect` (för att styra att anropet bara körs en gång vid sidstart) och `useState` (för att spara datan så att skärmen uppdateras).

```
import { useState, useEffect } from 'react';
import axios from 'axios'; // Importera axios efter installationen

function App() {
  const [produkter, setProdukter] = useState([]);
  const [laddar, setLaddar] = useState(true);
  const [felmeddelande, setFelmeddelande] = useState('');

  useEffect(() => {
    async function hämtaData() {
      try {
        // Axios gör anropet och packar upp JSON-datan direkt i .data
        const response = await axios.get('https://fakestoreapi.com');
        setProdukter(response.data);
      } catch (error) {
        // Om Axios möter t.ex. ett 404-fel hamnar vi direkt i catch
        setFelmeddelande("Kunde inte ladda produkterna.");
      } finally {
        setLaddar(false); // Laddningen är klar oavsett resultat
      }
    }
    hämtaData();
  }, []);

  if (laddar) return <h2>Laddar produkter...</h2>;
  if (felmeddelande) return <h2 style={{ color: 'red' }}>{felmeddelande}</h2>;

  return (
    <div>
      <h1>Vår Elektronikbutik</h1>
      <ul>
        {produkter.map(produkt => (
          <li key={produkt.id}><b>{produkt.title}</b> - {produkt.price} kr</li>
        ))}
      </ul>
    </div>
  );
}
```

```
    );  
  }  
  
  export default App;
```

Bild 7 Lägg till Axios i ett JavaScript-projekt, till exempel React JS.